

LECTURE 21

BCNF Schema Design by Decomposition

CSC270 · Database Systems

COMSATS University Islamabad, Lahore Campus

What We'll Cover

- The formal BCNF Decomposition Algorithm, stated as a repeatable loop
- How to identify which FD to split on when a relation has several violations
- A complete, multi-step worked example — iterated until every piece is in BCNF
- Why every split must be checked for the lossless-join property
- Dependency preservation: what it means, and why BCNF can sacrifice it
- The BCNF vs. 3NF synthesis trade-off, side by side

What BCNF Requires

A relation R is in Boyce-Codd Normal Form if, for every non-trivial functional dependency $X \rightarrow Y$ that holds over R , X is a superkey of R .

Every determinant must be able to determine ALL of R 's attributes — not just the ones on the right-hand side of that one dependency.

The BCNF Test (from Lecture 20)

Compute X^+ under R 's full FD set, and confirm X^+ covers every attribute of R . If some FD's determinant fails that test, R violates BCNF — and this lecture's algorithm tells you exactly how to fix it.

The BCNF Decomposition Loop

1. Pick a relation R

From the current decomposition — initially, just the original relation

2. Test R for BCNF

For every FD $X \rightarrow Y$ over R, check whether X^+ covers all of R's attributes

3. Split on a violating FD

Pick $X \rightarrow Y$ where X is NOT a superkey — a genuine violation

4. Replace R with R_1, R_2

$R_1 = X \cup Y$ $R_2 = X \cup (R - Y)$; project R's FDs onto each

5. Loop

Re-test R_1 and R_2 from step 1; stop only when every relation passes step 2

Easy to Get Wrong

1

X appears in both halves

This is not a mistake — it's exactly what makes the join lossless. X becomes the connecting column between R_1 and R_2 , playing the same role a foreign key plays after the split.

2

Re-test the NEW relations

Don't declare victory after one split. A single split can easily leave one or both halves still violating BCNF if the original relation had more than one problem FD — exactly what happens in the worked example next.

A Course Enrollment Relation

Enrollment (studentNo, courseNo, instructorNo, room)

studentNo	courseNo	instructorNo	room
S1001	CSC270	I01	LT-3
S1002	CSC270	I01	LT-3
S1001	CSC211	I02	LT-1
S1003	CSC270	I01	LT-3
S1003	CSC211	I02	LT-1

A university relation recording which student is enrolled in which course, taught by which instructor, meeting in which room.

The Business Rules (Functional Dependencies)

FD1: studentNo, courseNo $\rightarrow$ instructorNo, room

A student's enrollment in a course fixes who teaches it and where (a student can't be in two sections of the same course).

FD2: courseNo $\rightarrow$ instructorNo

Each course is taught by exactly one instructor (single-section policy — no parallel sections this term).

FD3: instructorNo $\rightarrow$ room

Each instructor teaches out of one fixed room all term.

Finding the Candidate Key

Before testing BCNF we need the candidate key. Try each attribute set's closure under FD1, FD2, FD3:

Attribute set tried	Closure	Reaches all 4 attributes?
studentNo	{studentNo}	No — no FD starts with studentNo alone
courseNo	{courseNo, instructorNo, room}	No — missing studentNo
{studentNo, courseNo}	{studentNo, courseNo, instructorNo, room}	Yes — via FD1 directly

Candidate key: {studentNo, courseNo}

Its closure already reaches all four attributes, and no smaller set does — this is the only legal superkey-forming determinant.

Testing the Original Relation

FD	Determinant	Is it a superkey of Enrollment?	BCNF?
FD1	studentNo, courseNo	Yes — it IS the key	OK
FD2	courseNo	No — $\text{courseNo}^+ = \{\text{courseNo}, \text{instructorNo}, \text{room}\}$	Violation
FD3	instructorNo	No — $\text{instructorNo}^+ = \{\text{instructorNo}, \text{room}\}$	Violation

Enrollment violates BCNF — twice over

Two different FDs (FD2 and FD3) each have a determinant that is not a superkey. The algorithm says: pick ONE violation and split; we'll pick FD2 first and come back to FD3's consequence afterward.

First Split — Setting Up the Split on FD2

Split on FD2: $\text{courseNo} \rightarrow \text{instructorNo}$. Set $X = \text{courseNo}$, $Y = \text{instructorNo}$.

$R_1 = X \cup Y = \{\text{courseNo}\} \cup \{\text{instructorNo}\}$
= {courseNo, instructorNo} → becomes CourseInstructor

$R_2 = X \cup (R - Y) = \{\text{courseNo}\} \cup \{\text{studentNo}, \text{courseNo}, \text{room}\}$
= {studentNo, courseNo, room} → becomes StudentCourseRoom

$X = \text{courseNo}$ appears in BOTH halves — it becomes the connecting column, exactly like the foreign key it will act as once these are separate tables.

First Split — the Resulting Relations

CourseInstructor (courseNo, instructorNo)

courseNo	instructorNo
CSC270	I01
CSC211	I02

StudentCourseRoom (studentNo, courseNo, room)

studentNo	courseNo	room
S1001	CSC270	LT-3
S1002	CSC270	LT-3
S1001	CSC211	LT-1
S1003	CSC270	LT-3
S1003	CSC211	LT-1

Not finished yet

CourseInstructor carries FD2 and looks fine. StudentCourseRoom still needs testing — it inherited FD3's consequence indirectly, as the next two slides show.

Testing CourseInstructor

CourseInstructor(courseNo, instructorNo) — the only FD projected here is FD2 itself: courseNo $\rightarrow$ instructorNo.

Check	Result
Candidate key	courseNo — its closure $\text{courseNo}^+ = \{\text{courseNo}, \text{instructorNo}\}$ reaches both attributes
Is the determinant a superkey?	Yes — courseNo IS the key

CourseInstructor is in BCNF — done

This relation needs no further splitting. Set it aside as one of the final pieces.

Why StudentCourseRoom Inherits a Violation

StudentCourseRoom(studentNo, courseNo, room) no longer even mentions instructorNo — so does FD3 (instructorNo $\rightarrow$ room) just vanish? No: its CONSEQUENCE survives indirectly, through transitivity.

Step A: courseNo $\rightarrow$ instructorNo

This is FD2, still true of the data.



Step B: instructorNo $\rightarrow$ room

This is FD3, still true of the data.

Chaining A and B: courseNo $\rightarrow$ room

By transitivity, courseNo determines room too — even though this new FD was never written down explicitly; it's a logical consequence of FD2 + FD3.

Testing StudentCourseRoom — the Inherited Violation

Now test courseNo → room against StudentCourseRoom's candidate key, {studentNo, courseNo}:

Check	Result
courseNo ⁺ over StudentCourseRoom	{courseNo, room} — using the inherited FD
Does it cover studentNo?	No — studentNo is missing
Is courseNo a superkey here?	No — Violation

StudentCourseRoom violates BCNF — split again

This is exactly the “repeat” in “iterative BCNF decomposition.” The first split fixed FD2's violation but inherited FD3's problem into one half, which needs its own second pass.

STEP 4

Second Split — on courseNo → room

Splitting StudentCourseRoom on courseNo → room. X = courseNo, Y = room.

CourseRoom (courseNo, room)

courseNo	room
CSC270	LT-3
CSC211	LT-1

StudentCourse (studentNo, courseNo)

studentNo	courseNo
S1001	CSC270
S1002	CSC270
S1001	CSC211
S1003	CSC270
S1003	CSC211

StudentCourse is all-key

{studentNo, courseNo} — no non-trivial FDs remain, so it is automatically in BCNF.

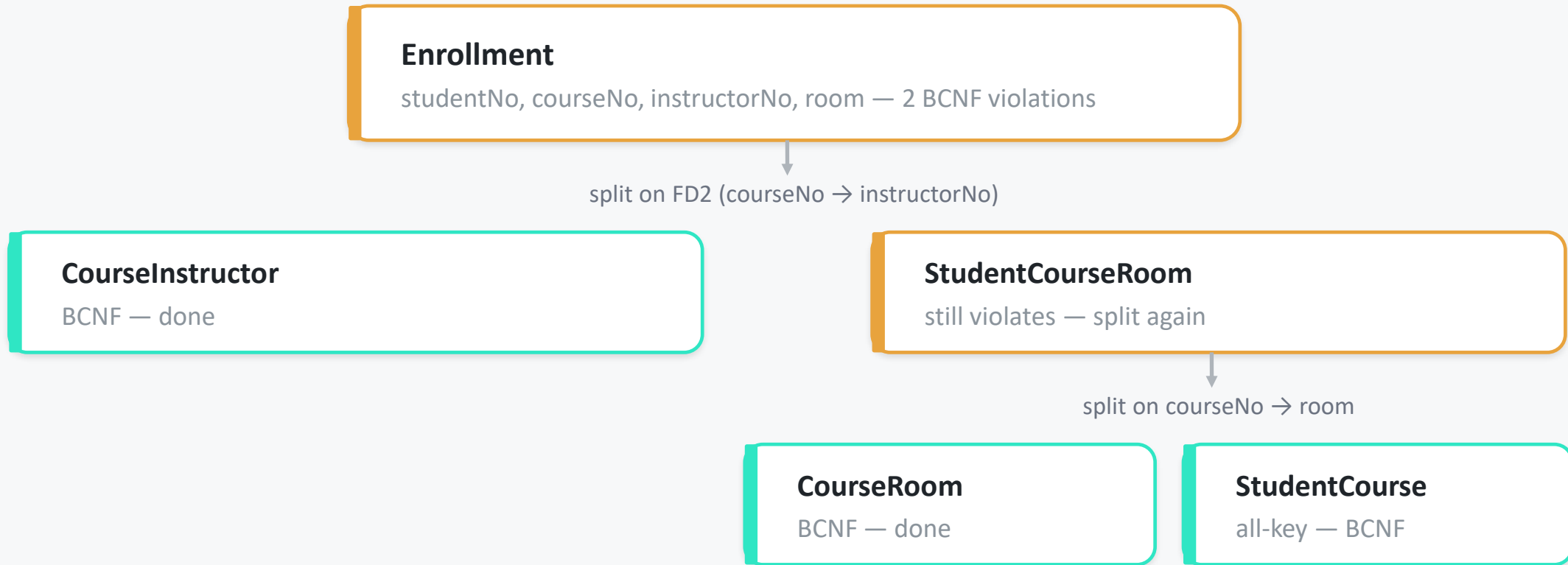
Final BCNF Check on Every Piece

Relation	Candidate key	Every determinant a superkey?	BCNF?
CourseInstructor(courseNo, instructorNo)	courseNo	Yes	Pass
CourseRoom(courseNo, room)	courseNo	Yes	Pass
StudentCourse(studentNo, courseNo)	studentNo, courseNo	Yes — all-key relation	Pass

Three relations, not two

A direct consequence of Enrollment having had two independent violations, not one. The first split fixed FD2's violation but inherited FD3's problem into one half, which needed a second pass to resolve.

The Full Decomposition Tree



One split can require more than one pass

Two violations in the original relation $\rightarrow$ two passes of the loop $\rightarrow$ three final relations.

Checking the Lossless-Join Property

A decomposition is lossless if natural-joining the pieces back together reconstructs exactly the original relation — no rows lost, no spurious rows gained.

Theorem (Lecture 20): a decomposition of R into R_1, R_2 is lossless if and only if $R_1 \cap R_2 \rightarrow R_1$ or $R_1 \cap R_2 \rightarrow R_2$ holds.

Every BCNF-algorithm split satisfies this automatically

Here $R_1 \cap R_2 = X$, and $X \rightarrow Y$ was the very FD we split on — so $X \rightarrow R_1$ holds trivially ($R_1 = X \cup Y$, and $X \rightarrow X \cup Y$ whenever $X \rightarrow Y$). You never need to separately prove losslessness for a BCNF-algorithm split; you only need to confirm you split on a real, verified FD.

Lossless-Join, Checked by Hand

StudentCourse ⋈ CourseInstructor ⋈ CourseRoom (on courseNo, then studentNo+courseNo)

studentNo	courseNo	instructorNo	room
S1001	CSC270	I01	LT-3
S1002	CSC270	I01	LT-3
S1001	CSC211	I02	LT-1
S1003	CSC270	I01	LT-3
S1003	CSC211	I02	LT-1

Five rows out, five rows in

Every original Enrollment tuple reappears exactly once, with no extras. Lossless does not mean “nothing changed” — it means “nothing lost or invented.”

Dependency Preservation — the Definition

A decomposition is dependency-preserving if every FD from the original relation's FD set either holds directly within one decomposed relation, or can be inferred by combining FDs that hold within the decomposed relations — without ever computing a join just to check a constraint.

3NF synthesis always preserves every original FD. BCNF decomposition sometimes cannot — even when it is fully lossless.

Why this matters

If a dependency is lost, the DBMS can no longer enforce that business rule with a simple PRIMARY KEY or UNIQUE constraint on one small table — the next slide shows exactly where that happens in our example.

Where the Loss Actually Happens

FD2 (courseNo → instructorNo) lives entirely inside CourseInstructor — fully preserved, enforceable with a key on that one table.

FD3 (instructorNo → room) is the one that gets hurt

It's only implied by combining CourseInstructor + CourseRoom. No single table stores instructorNo and room together anymore, so no single table's key can check it.

Concrete insert anomaly:

- INSERT INTO CourseInstructor VALUES ('CSC401', 'I01') — legal: courseNo is the key, no violation locally
- INSERT INTO CourseRoom VALUES ('CSC401', 'LT-5') — also legal, same reasoning
- Together they imply I01 teaches from BOTH LT-3 and LT-5 — FD3 is now broken, and neither table's own key constraint could ever catch it

Three Practical Choices

This is a real design decision, not a footnote

When a BCNF decomposition loses dependency preservation, you have three practical choices in a real system.

1

Accept BCNF

Enforce the lost constraint with a database trigger or application-level check instead of a simple key.

2

Settle for 3NF

Deliberately tolerate a small amount of redundancy on that one relation, in exchange for a locally checkable constraint.

3

Redesign the rule

If the lost dependency turns out not to be as important as it first seemed, change the business rule itself.

3NF Synthesis vs. BCNF Decomposition

3NF Synthesis

Always lossless AND always dependency-preserving — the synthesis algorithm builds one relation per FD specifically to guarantee this.

Trade-off: may retain a small amount of redundancy that BCNF would remove.

BCNF Decomposition

Always lossless, but NOT guaranteed dependency-preserving.

Trade-off: removes all redundancy tied to a non-key determinant, at the possible cost of needing a join to re-check some constraint.

Testing Relations for BCNF — the Checklist

- 1 List every attribute of the relation and every FD that holds over it (project the original FD set onto just this relation's attributes)
- 2 Find every candidate key by computing closures (Lecture 20)
- 3 For every non-trivial FD $X \rightarrow Y$ in that projected set, compute X^+
- 4 If X^+ covers ALL attributes, that FD is fine; if even one FD fails this test, split on it and repeat on both halves
- 5 Stop only when every relation currently in the decomposition passes with no exceptions

Key Takeaways

- The BCNF algorithm is a loop: while some relation violates BCNF, split on a violating FD $X \rightarrow Y$ into $R_1 = X \cup Y$ and $R_2 = X \cup (R - Y)$, then re-test both halves.
- A relation can have more than one BCNF violation — fixing one split can leave or inherit a second, exactly as Enrollment needed two passes.
- Every split by this rule is automatically lossless: $R_1 \cap R_2 = X$ and $X \rightarrow R_1$ holds by construction — no separate proof needed.
- Unlike 3NF synthesis, BCNF decomposition is NOT guaranteed to preserve every original FD — a genuine trade-off worth naming explicitly in a design review.
- The re-test-every-piece discipline, driven by closure computation, is the engine of the whole algorithm — there is no shortcut around it.